Armageddon 6.5

The Brotherhood of Evil jerMutants - Wolfpack

Authors:

John Sweeney

T.I.Q.S.

Michael Dale

Cassiem Davids

Vincent Woods – Uriah Victorious

Benji

Royce Billot

repos:

Final Project - Armageddon repo: https://github.com/jastek69/gcp-ha-vpn-hub-spokes-SA-scaffolding.git

Group workspace repo: https://github.com/jastek69/gcp-ha-vpn-hub-spokes

Task 3 Submissions:

Michael Dale Armageddon Task 3 Repo: https://github.com/MichaelDale1/GCP-Armageddon-2025-Dale/tree/Armageddon_Task_3

Cassiem Armageddon Task 3 Repo: https://github.com/kazapes/GCPArmageddon2025-cass.git

Uriah Victorious Armageddon Task 3 Repo: https://github.com/UriahVictorious/gcp-be-a-man-level-5

Benji Armageddon Task 3 Repo: https://github.com/Newest-IT-Bro/Benji-Armageddon-Task-3-Repo.git

Royce Billiot Armageddon Task 3 Repo: https://github.com/Rbilli51614/GCP-Armageddon-Task-3.git

This network design utilizes account impersonation in a Hub and Spoke infrastructure built using terraform scaffolding repositories.

Global Enterprise-Level Infrastructure contains the following

- Multi-region VMs across the globe
- Cross-project VPN tunnels with HA
- BGP peering between all spokes
- Custom IAM roles and cross-project permissions (no need for json files)
- Network Connectivity Hub integration

Summary:

- 6-project global infrastructure spanning 4 continents (Asia, North America, South America, Europe)
- Complete hub-spoke VPN mesh with BGP routing for full connectivity Terraform scaffolding to handle proper bootstrap/infrastructure separation
- Service account impersonation for secure multi-project deployment
- Automated scripts that handle the complexity seamlessly
- Modular architecture that's easily extensible

This project uses scaffolding dividing the repository into two primary areas decoupling the authentication process from the building of the network framework:

- Bootstrap: handles the provisioning and authentication seeding the IAM roles granting the Service accounts permissions for Account Impersonation in order allow resources to be built with terraform. This will only need to be run at the start of a project or when a new project is to be added to the infrastructure.
- Infra (infrastructure): handles the building of the compute and network resources. Once bootstrapping has been completed and the proper roles have been provided the infrastructure can be built or destroyed without the need to remove the permissions that have been established.

When adding new accounts to expand the network it is necessary to run Bootstrap first and then add the new network resources through Infra.

Figure 1: terraform bootstrap mapping

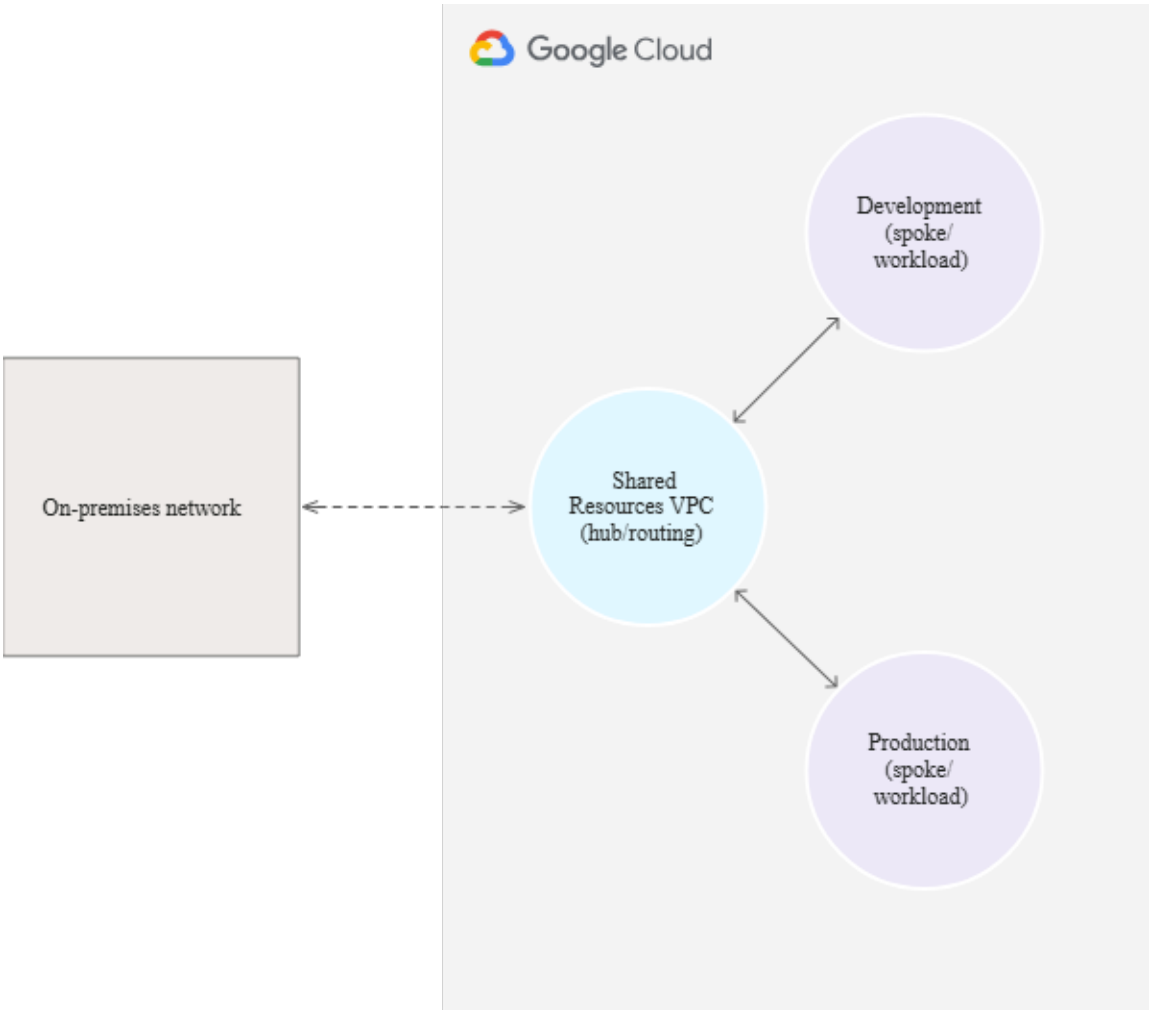
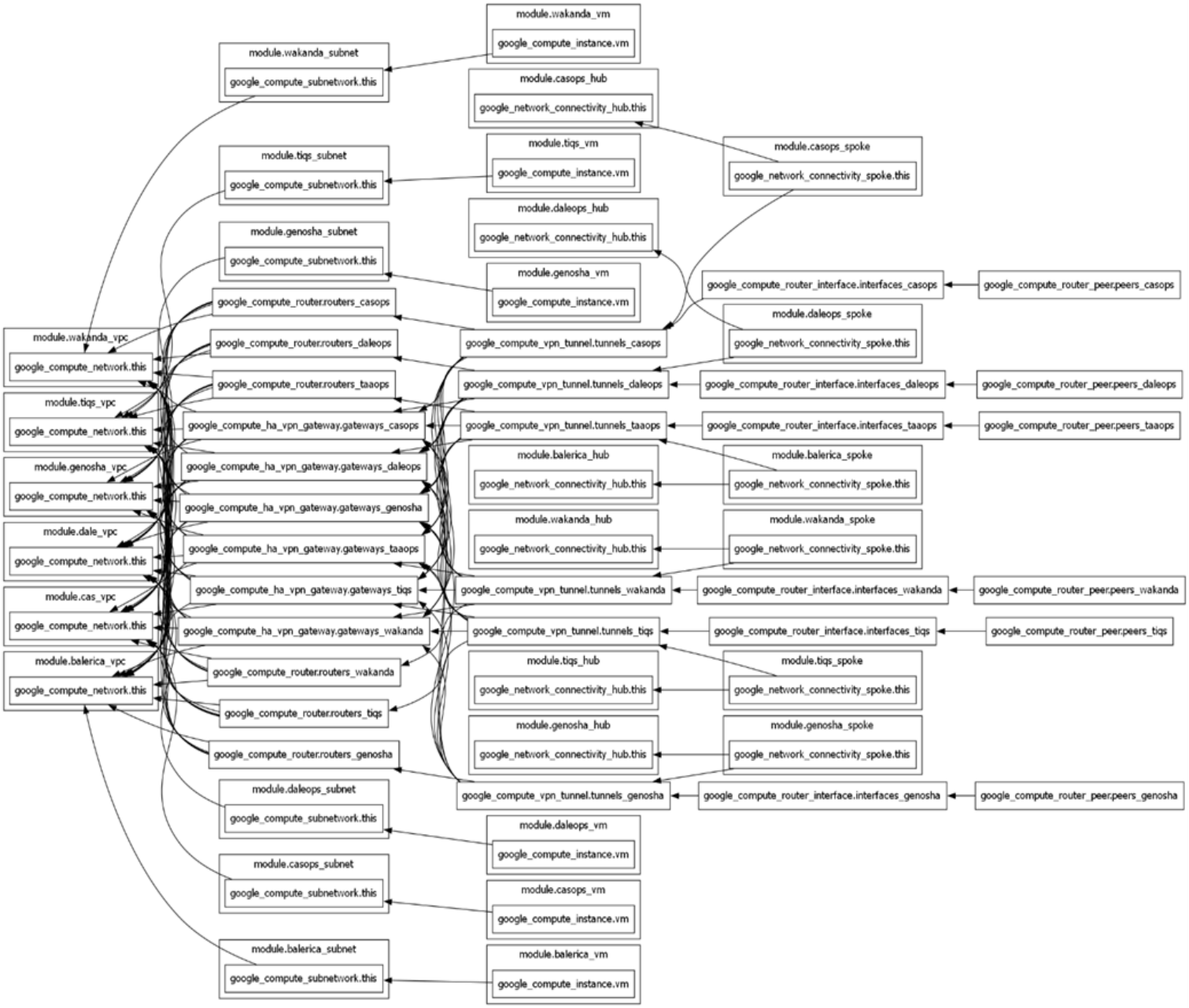


Figure 3 - Terraform Infra Mapping



Hub-and-Spoke VPN Terraform Setup

This repository provisions a hub-and-spoke HA VPN environment in Google Cloud using Scaffold Terraform modules with env and variables. It includes projects, Modules, IAM roles, HA VPN gateways, tunnels, Cloud Routers tf files, and startup scripts. This is a secure configuration with separate deployments for bootstrap mode and an infrastructure mode. Variables are set inside .tfvars and .env so there is no need for a JSON file. It is also configured to be scalable as well as being deployable within CI/CD pipelines. This can be easily done by setting the account within .tfvars to either the CI/CD account the user account.

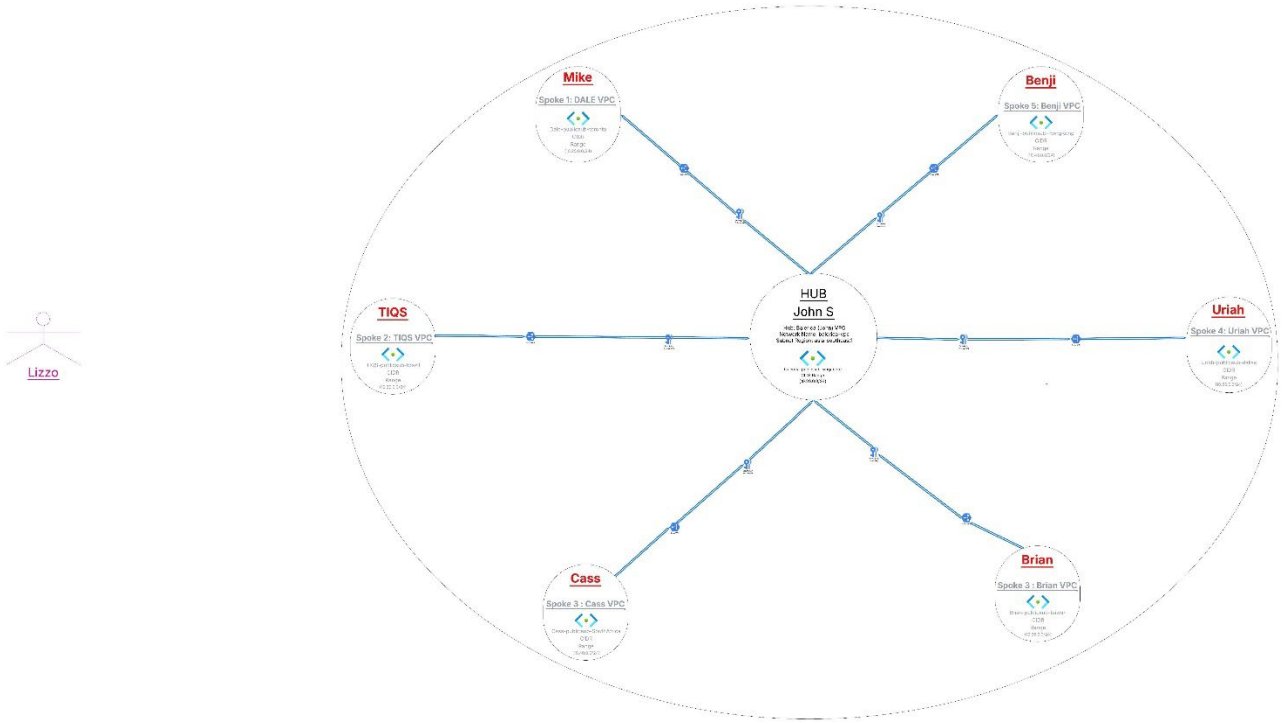


Figure 3: Network Flowchart

Resources Used:

- Authentication Methods at Google: https://cloud.google.com/docs/authentication
- Service Account Impersonation: https://cloud.google.com/iam/docs/service-account-impersonation

- google_service_account: https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/google_service_account_iam
- **IAM policy for Service Account**

  When managing IAM roles, you can treat a service account either as a resource or as an identity. This resource is to add iam policy bindings to a service account resource, such as allowing the members to run operations as or modify the service account. To configure permissions for a service account on other GCP resources, use
  the [google_project_iam](https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/google_project_iam) set of resources.

  Three different resources help you manage your IAM policy for a service account. Each of these resources serves a different use case:

  - **google_service_account_iam_policy:** https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/google_service_account_iam#google_service_account_iam_policy-1 Authoritative. Sets the IAM policy for the service account and replaces any existing policy already attached.
  - **google_service_account_iam_binding:**
  - (https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/google_service_account_iam#google_service_account_iam_binding-1): Authoritative for a given role. Updates the IAM policy to grant a role to a list of members. Other roles within the IAM policy for the service account are preserved.

  - **google_service_account_iam_member`**](https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/google_service_account_iam#google_service_account_iam_member-1): Non-authoritative. Updates the IAM policy to grant a role to a new member. Other members for the role for the service account are preserved.

    Note:

google_service_account_iam_policy cannot be used in conjunction with **google_service_account_iam_binding** and **google_service_account_iam_membe** or they will fight over what your policy should be.

google_service_account_iam_binding resources **can be** used in conjunction with g**oogle_service_account_iam_member** resources **only if** they do not grant privilege to the same role.

- Scaffolding: https://jiminbyun.medium.com/platform-engineering-starter-kit-series-repository-scaffolding-minimal-infra-bootstrap-2-5a9034444219

- CODEX and Github CoPilot

Project Details

Service Account Impersonation

How this works:

- When you run Terraform, the active gcloud user (ADC) is used by default.
- If var.balerica_sa or var.genosha_sa are set, Terraform will impersonate those service accounts using your ADC credentials.
- This avoids ever downloading or storing JSON keys - No JSON files are needed locally.
- In CI/CD, you can do the same with Workload Identity Federation — no keys, just impersonation.
- You authenticate with gcloud auth login (or gcloud auth application-default login).
- Terraform uses your/ active credentials to impersonate the service accounts in each project.
- If you later rotate or recreate the service account keys, nothing changes in Terraform — it always impersonates.

Security

The design utilizes the concept of Least Privilege through IAM Roles and Polices. It slos dow not require a JSON key and instead uses Account Impersonation. It can also easily be implemented with CI/CD pipeline though the use of service accounts.

- No JSON keys.
- You can grant temporary impersonation rights to human users or CI/CD identities.
- The impersonated SA is what Terraform actually uses to create VPN, routers, IAM, etc.

Utilizing the service account for terraform (as opposed to Compute) through Service Account Impersonation eliminates the need for a JSON (the use of service account keys). See documentation: https://cloud.google.com/iam/docs/service-account-impersonation

You can use service account impersonation to set up a local Application Default Credentials (ADC) file. Client libraries that support impersonation can use those credentials automatically. Local ADC files created by using impersonation are supported in the following languages:

- C#
- Go
- Java
- Node.js
- Python

1. Install Requirements

- Terraform: ≥ 1.3
- gcloud CLI: gcloud auth application-default login
- Project APIs, Service Accounts with Application Default Credentials: see below

2. Prerequisite: Service Accounts Details

Service accounts need the proper IAM permissions in order to be able to build the infrastructure. The primary user account that is running terraform needs permission

granted from the source project that is to be added to the infrastructure. To ensure the proper roles have been provided utilize the following commands. Once bootstrap has been completed then these permissions can be removed.

Service Account Permissions

Before you begin, any Project using this method will require the proper API's and a Terraform Service Account configured with the proper Roles. The following API's and permissions need to be granted to the Project and service account:

Project API's

The project will need the following APIS enabled:

- Network Connectivity API
- Google Cloud API
- Identity and Access Management (IAM) API
- Security Token Service API

Terraform Service Account Roles - Permissions needed for the operator

- *Owner - DO NOT USE*- to follow the concept of Least Privilege it is strongly recommended not to grant Owner permission. Instead grant the following Roles:
- Artifact Registry Admin
- Storage Admin
- Project IAM Admin
- roleAdmin / iam.roleAdmin - for Role Administrator
- iam.serviceAccountUser
- resourcemanager.projectIamAdmin
- serviceusage.serviceUsageAdmin
- artifactregistry.admin
- networkconnectivity.admin
- iap.tunnelResourceAccessor
- Service Account Token Creator
- compute.admin
- compute.instanceAdmin.v1
- compute.networkAdmin
- compute.osLogin

- compute.viewer
- viewer

Service Account Verification

To verify the Service Account has the proper permissions granted by a project that is to be added run the following checks.

Login with Default credentials

`gcloud auth application-default login`

Quick permission checks

to view a list the service account roles on a project:

gcloud projects get-iam-policy <project> --flatten="bindings[].members" --filter="bindings.members:user:<account>" --format="table(bindings.role, bindings.members)"

Ensure SA token creator (example for taaops Terraform SA):

gcloud iam service-accounts get-iam-policy terraform@taaops.iam.gserviceaccount.com --project=taaops --format='table(bindings.role, bindings.members)'

Verify project permissions:

gcloud projects get-iam-policy <project> \

--flatten="bindings[]" \

--format="table(bindings.role, bindings.members)" \

--filter="bindings.members:user:you@example.com"

Complete listing of account permissions:

gcloud projects get-iam-policy <project name> \

```
 --flatten="bindings[].members" \
 --filter="bindings.members:user:<account@example.com>" \
 --format="table(bindings.role, bindings.members)"
```

Check projects for roles:

```
gcloud iam service-accounts get-iam-policy
terraform@<project1>.iam.gserviceaccount.com \
 --project=<project1> --format='table(bindings.role, bindings.members)'

gcloud iam service-accounts get-iam-policy
terraform@<project2>.iam.gserviceaccount.com \
 --project=<project2> --format='table(bindings.role, bindings.members)'

gcloud iam service-accounts get-iam-policy jasops-
terraform@<project3>.iam.gserviceaccount.com \
 --project=<project3> --format='table(bindings.role, bindings.members)'

gcloud iam service-accounts get-iam-policy terraform-
service@<project4>.iam.gserviceaccount.com \
 --project=<project4> --format='table(bindings.role, bindings.members)'

gcloud iam service-accounts get-iam-policy terraform-service-
delta@<project5>.iam.gserviceaccount.com \
 --project=<project5> --format='table(bindings.role, bindings.members)'
```

3. Deployment

Scripts

In order ease the use of steps required to properly deploy this environment using scaffolding a Bootstrap script ` ./scripts/bootstrap.sh` and Infra (Infrastucture) ` ./scripts/infra.sh` can be used instead of manually going through the bootstrapping.

- ***boostrap.sh*** - Seeds IAM bindings. Runs the initial bootstrap granting permission for Account Impersonation
- ***infra.sh*** - creating and tearing down of the network infrastructure

NOTE: These commands are intended to be run from the root directory:

bootstrap.sh

- Seeds IAM bindings/custom roles across projects.
- Apply: ` ./scripts/bootstrap.sh apply`
- Destroy (optional; wipes bootstrap IAM/service accounts): ` ./scripts/bootstrap.sh destroy`. Only use this when completely destroying everything and accounts are no longer needed.
- Run `bootstrap.sh` first when creating the infrastructure or when adding new resources from outside accounts.

infra.sh

- Builds compute + networking
- Apply with tfvars and bootstrap_mode=false: ` ./scripts/infra.sh apply`
- Destroy compute/network only (keeps bootstrap IAM intact): ` ./scripts/infra.sh destroy`

Option - Run from Bootstrap or Infra directory

Alternatively, these scripts can also be run from within each directory. In order to deploy it properly using the configuration settings from `tfvars` and `.env` you need instruct terraform to reference `.tfvars` when running both the scripts (bootstrap or infra): `-var-file=../terraform.tfvars`

example: `terraform plan -var-file=../terraform.tfvars -out=tfplan_bootstrap`

- The scripts can be run from the root directory or you can also change to the appropriate directory .
- Once inside bootstrap or infra directory run valiadate, plan, apply while calling tfvars:
- `terraform plan -var-file=../terraform.tfvars -out=tfplan_bootstrap`
- With this method you can also point to an alternative .tfvars file

MAKEFILE

There is also a Makefile which can be used to deploy the infrastructure. In order to use the Makefile be sure:

- terraform CLI is installed
- repo root contains terraform.tfvars so the Makefile/scripts is able to use it or set TFVARS=/path/to/tfvars.

How to run Makefile:

Bootstrap (IAM/custom roles, one-time setup):

- ***make bootstrap-plan*** - to preview
- ***make bootstrap*** - to apply terraform and build bootstrap
- ***make bootstrap-destroy*** - to tear down bootstrap only

Infra (compute/network):

- ***make infra-plan*** - preview
- ***make infra*** - to apply terraform and build the networking and compute infrastructure

- ***make infra-destroy*** - destroy infra only; (bootstrap IAM stays)

Full teardown:

- ***make destroy-all*** - runs infra-destroy then bootstrap-destroy

Cleanse:

- ***make fmt*** - to format
- ***make validate*** - to validate both stacks
- ***make clean-plans*** - to delete saved plan files